Subject: Machine Learning1 **Level:** Master	PRISTINI SCHOOL OF AI	**Academic year:** 2026/2027 **A.Prof**: Ameni YOUSSFI

Lab n ° 1

Datamining and Machine learning

Objective:

In this Lab, you will practice applying linear regression techniques to analyze a dataset and build a predictive model. You will explore relationships between multiple independent variables and a dependent variable, evaluate the model's performance, and interpret the results.

Dataset:

You can download the dataset from the following link: https://www.kaggle.com/datasets/prokshitha/home-value-insights?resource=download

Instructions :

1. Download the dataset and load it into a Pandas DataFrame using Python in Google Colab.
2. Explore the dataset to understand its structure and identify the target variable (price) and potential independent variables.

Tasks:

Part 1: Data Understanding and Cleaning

1. Load the dataset and display the first few rows.
2. Check for missing values.
3. Describe the dataset: number of rows, columns, data types, and basic statistics.

Let's begin!

Module 1: Introduction

In this section you will learn how to approach data acquisition in various ways, and obtain necessary insights from a dataset. By the end of this section, you will successfully load the House Price Regression dataset into Jupyter Notebook / Google Colab, and gain some fundamental insights via the Pandas Library.

Table of Contents

- Data Acquisition
- Basic Insight of Dataset

1. Data Acquisition

There are various formats for a dataset .csv, .json, .xlsx, etc. The dataset can be stored in different places, on your local machine or sometimes online. In this section, you will learn how to load a dataset into your Jupyter Notebook.

In our case, the House Price Regression dataset is stored in CSV (comma separated value) format. It contains 1,000 records describing residential properties with 8 features used to predict house price.

The Pandas Library is a useful tool that enables us to read various datasets into a data frame. All we need to do is import Pandas without installing additional packages.

Dataset Columns

Column Name	Data Type	Description
Square_Footage	int64	Total area of the house in sq. ft.
Num_Bedrooms	int64	Number of bedrooms (1–5)
Num_Bathrooms	int64	Number of bathrooms
Year_Built	int64	Year the house was constructed
Lot_Size	float64	Size of the lot in acres
Garage_Size	int64	Number of cars the garage can hold
Neighborhood_Quality	int64	Quality score of neighborhood (1–10)
House_Price	float64	Target variable — sale price in USD

Read Data

We use the pandas.read_csv() function to read the csv file. In the brackets, we put the file path along with a quotation mark so that pandas will read the file into a data frame.

```
# import pandas library
import pandas as pd
#upload file on google colab
```

```
from google.colab import files
uploaded = files.upload()


# read the dataset and assign it to variable 'df'
df = pd.read_csv('house_price_regression_dataset.csv')
print('Done')
```

After reading the dataset, we use the dataframe.head(n) method to check the top 'n' rows, and dataframe.tail(n) to show the bottom rows.

```
# show the first 5 rows using dataframe.head() method
df.head(5)
```

Question #1:
Check the bottom 10 rows of the data frame 'df'.

Answer #1:

```
df.tail(10)
```

Our dataset has no pre-defined headers issues since it already includes a header row. Let's confirm the shape of the dataset:

```
df.shape
```

Expected output: (1000, 8) — 1,000 rows and 8 columns.

Question #2:
Find the name of the columns of the dataframe 'df'.

Answer #2:

```
df.columns
```

Basic Insight of Dataset

After reading data into a Pandas dataframe, it is time to explore the dataset. There are several ways to obtain essential insights of the data to help us better understand it.

Data Types

Data has a variety of types. The main types stored in Pandas dataframes are object, float, int, bool, and datetime64. In order to better learn about each attribute, it is good practice to know the data type of each column.

```
# check the data type of data frame 'df' by .dtypes
```

```
df.dtypes
```

As you can see, Square_Footage, Num_Bedrooms, Num_Bathrooms, Year_Built, Garage_Size, and Neighborhood_Quality are int64, while Lot_Size and House_Price are float64. No object columns exist ➔ all features are numeric, which is ideal for regression.

Describe

If we would like to get a statistical summary of each column, such as count, mean value, standard deviation, etc., we use the describe method:

```
df.describe()
```

This provides summary statistics for all numeric columns. For example, House_Price has 1,000 records, a mean of approximately $618,861, standard deviation of $253,568, minimum of $111,627, and maximum of $1,108,237.

```
# describe all the columns in 'df' (categorical columns)
df.describe(include = 'all')
```

Question #3:
Apply the .describe() method to the columns 'Square_Footage' and 'Year_Built'.

Answer #3:

```
df[['Square_Footage', 'Year_Built']].describe()
```

Info

Another method to check your dataset is dataframe.info(). It provides a concise summary of your DataFrame including data types, non-null counts, and memory usage.

```
# look at the info of 'df'
df.info()
```

The output confirms the dataframe has 1,000 rows and 8 columns, with no null values — this is a clean dataset.

Module 2: Data Wrangling

By the end of this section, you will have learned the basics of Data Wrangling!

Table of Contents

- Identify and handle missing values
- Data standardization
- Data Normalization (centering/scaling)
- Indicator variable (dummy variable)

Task to do: download and display all information about this dataset: https://www.kaggle.com/datasets/ghost5/messy-house-price-sample-datasets

What is the purpose of Data Wrangling?

Data Wrangling is the process of converting data from its initial format to a format that may be better for analysis. Even a clean dataset like the House Price dataset may require transformations to make it suitable for machine learning models.

```
# To see what the data set looks like, we use the head() method.
df.head()
```

1. Identify and Handle Missing Values

Check for Missing Values

Convert "unavailable" to NaN

In this dataset, some missing data comes with the placeholder string "unavailable" (found in the 'price' column). We replace "unavailable" with NaN (Not a Number), which is Python's default missing value marker, for reasons of computational speed and convenience. Here we use the function:

`.replace(A, B, inplace=True)`

to replace A by B.

```
import numpy as np

# replace 'unavailable' with NaN in the price column
df['price'].replace('unavailable', np.nan, inplace=True)
df.head(5)
```

Check for Missing Values

The missing values are now converted to Python's default NaN. We use Python's built-in functions to identify them. There are two methods to detect missing data:

- .isnull() — returns True where data is missing
- .notnull() — returns True where data is present

Run the following to count missing values per column:

```
df.isnull().sum()

# Expected output:
# area_sqft      0
# bedrooms      21
# bathrooms      0
# year_built    36
# price         50
# location       0
```

Dealing with Missing Data

General strategy:

- Replace by frequency or based on other functions or mean, for continuous numeric columns (bedrooms, year_built)
- Drop rows or the whole column, when the target variable (price) is missing.

The whole column should be dropped only if most entries in the column are empty
(this is not the case in our dataset)

Example: replace NaN with mean for the 'bedrooms' column

```
import numpy as np

avg_bed = df['bedrooms'].astype('float').mean(axis=0)
df['bedrooms'].replace(np.nan, avg_bed, inplace=True)
```

Question #1: According to the example above, write the code to replace NaN values in the 'year_built' column with the mean.

Answer #1:

```
avg_year = df['year_built'].astype('float').mean(axis=0)
df['year_built'].replace(np.nan, avg_year, inplace=True)
```

Drop rows where price is missing (target variable):

```
df.dropna(subset=['price'], axis=0, inplace=True)
df.reset_index(drop=True, inplace=True)
```

To see which values are present in a particular column, we can use the ".value_counts()" method:

```
df['year_built'].value_counts()
```

Nice work! Now we obtain the dataset with no missing values.

Correct Data Format

Verify and fix data types:

```
df.dtypes

# Fix: ensure numeric columns are float
df[['area_sqft', 'bedrooms', 'bathrooms']] = df[['area_sqft', 'bedrooms',
'bathrooms']].astype('float')
df[['price']] = df[['price']].astype('float')
print('Done')
```

Also drop or correct any invalid values, for example, area_sqft cannot be negative:

```
# Drop rows with negative area_sqft
df = df[df['area_sqft'] > 0]
df.reset_index(drop=True, inplace=True)
```

➔ Nice! Now we finally obtain the cleaned dataset with no missing values and all data in its proper format

2. Data Standardization

Data standardization transforms data into a consistent format that enables meaningful comparison.

Data Standardization is the process of converting data into a **common and consistent format** so that values can be compared fairly and used effectively by machine learning algorithms.

Example: Deriving House Age from year_built

The column 'year_built' records the year a house was built. We derive 'house_age' by subtracting from the current year:

```
import datetime

current_year = datetime.datetime.now().year
df['house_age'] = current_year - df['year_built']

df.head()
```

Standardizing Location Labels

The 'location' column contains many variants of the same city (e.g. 'SF', 'sf', 'San Francisco', 'SAN FRANCISCO'). We standardize them to a canonical form:

```
city_map = {
    'sf': 'San Francisco', 'SF': 'San Francisco', 'Sf': 'San Francisco',
    'SAN FRANCISCO': 'San Francisco',
    'nyc': 'New York City', 'NYC': 'New York City', 'Nyc': 'New York City',
```

```
    'New York city': 'New York City', 'NEW YORK CITY': 'New York City',
    'la': 'Los Angeles', 'LA': 'Los Angeles', 'La': 'Los Angeles',
    'l.a.': 'Los Angeles', 'L.A.': 'Los Angeles',
    'LOS ANGELES': 'Los Angeles', 'los angeles': 'Los Angeles',
    'austin': 'Austin', 'AUSTIN': 'Austin',
    'chicago': 'Chicago', 'Chicgo': 'Chicago', 'CHICAGO': 'Chicago',
    'miami': 'Miami', 'MIAMI': 'Miami',
    'boston': 'Boston', 'Bostan': 'Boston', 'BOSTON': 'Boston',
}

df['location'] = df['location'].replace(city_map)
df['location'].value_counts()
```

Question #2: According to the example above, create a new column 'price_per_sqft' by dividing 'price' by 'area_sqft'.

Answer #2:

```
df['price_per_sqft'] = df['price'] / df['area_sqft']
df.head()
```

3. Data Normalization

Why Normalization?

Normalization scales features into a similar range so that no single feature dominates model training. For example, 'area_sqft' ranges from ~100 to 5,000 while 'bedrooms' ranges from 1 to 20 — very different scales.

- Prevent large-scale features from dominating smaller ones
- Speed up gradient descent convergence
- Improve performance of distance-based algorithms
- Reduce the impact of different units of measurement

Main Techniques

- **Min-Max Scaling**Rescales values to a fixed range, typically [0, 1]:

$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Standardization (Z-score)**Centers data around 0 with a standard deviation of 1:

$$X' = \frac{X - \mu}{\sigma}$$

- **Robust Scaling** Uses median and IQR — resistant to outliers:

$$X' = \frac{X - \text{median}}{IQR}$$

- **Max Abs Scaling**Divides by the maximum absolute value → range [-1, 1]:

$$X' = \frac{X}{|X_{max}|}$$

- **L2 Normalization (Unit Norm)** Scales each sample vector to unit length (used in NLP, clustering):

$$X' = \frac{X}{||X||_2}$$

When to Use Each Technique?

Technique	Best For
Min-Max	Neural networks, image pixel data
Z-score	Linear models, logistic regression
Robust Scaling	Data with significant outliers
Max Abs Scaling	Sparse matrices (e.g. TF-IDF)
L2 Normalization	NLP, cosine similarity tasks

We apply Min-Max Normalization:

```
# Normalize 'area_sqft' and 'price_per_sqft'
df['area_sqft'] = df['area_sqft'] / df['area_sqft'].max()
df['price_per_sqft'] = df['price_per_sqft'] / df['price_per_sqft'].max()
```

Question #3: According to the example above, normalize the column 'price' so values range from 0 to 1.

Answer #3:

```
df['price'] = df['price'] / df['price'].max()

# Verify scaled columns
df[['area_sqft', 'price_per_sqft', 'price']].head()
```

- We can use also z_score

```
# Z-score normalization for 'area_sqft' and 'price_per_sqft'
df['area_sqft'] = (df['area_sqft'] - df['area_sqft'].mean()) / df['area_sqft'].std()
```

```
df['price_per_sqft'] = (df['price_per_sqft'] - df['price_per_sqft'].mean()) / df['price_per_sqft'].std()


# Answer #3: Normalize 'price' using Z-score
df['price'] = (df['price'] - df['price'].mean()) / df['price'].std()


# Verify scaled columns
df[['area_sqft', 'price_per_sqft', 'price']].head()
```

4. Indicator Variables (Dummy Variables)

What is an Indicator Variable?

An indicator variable (dummy variable) is a numerical variable used to represent categories. We use them so that categorical data can be used in regression analysis.

Example: Encoding location

After standardizing the location labels, we can create one-hot encoded dummy variables:

```
import pandas as pd

dummy_location = pd.get_dummies(df['location']).astype(int)
dummy_location.head()
```

```
# Merge back into the dataframe and drop the original column
df = pd.concat([df, dummy_location], axis=1)
df.drop('location', axis=1, inplace=True)
df.head()
```

Example: Binning house_age

We can also create categories from numeric columns. The 'house_age' values are grouped into three bands:

```
def age_label(age):
    if age <= 20:  return 'new'
    elif age <= 60: return 'mid'
    else:           return 'old'

df['age_category'] = df['house_age'].apply(age_label)
df['age_category'].value_counts()
```

```
dummy_age = pd.get_dummies(df['age_category']).astype(int)
df = pd.concat([df, dummy_age], axis=1)
df.drop('age_category', axis=1, inplace=True)
df.head()
```

Question #4: Create indicator variables for a new column 'bathroom_category' where bathrooms <= 2 is 'few_bathrooms' and bathrooms > 2 is 'many_bathrooms'. Merge it back into the dataframe and drop 'bathroom_category'.

Answer #4:

```
df['bathroom_category'] = df['bathrooms'].apply(
    lambda x: 'few_bathrooms' if x <= 2 else 'many_bathrooms'
)
dummy_bath = pd.get_dummies(df['bathroom_category']).astype(int)
df = pd.concat([df, dummy_bath], axis=1)
df.drop('bathroom_category', axis=1, inplace=True)
df.head()
```

Question #5: Display the first 5 rows of the final dataframe and verify that all new indicator columns are present and the original categorical columns have been removed.

Answer #5:

```
print(df.columns.tolist())
df.head()
```

Save the Cleaned Dataset

Save the final cleaned and enriched dataframe to CSV:

```
from google.colab import files
files.download('clean_housing_data.csv')
```

Read / Save Other Data Formats

Format	Read	Save
csv	`pd.read_csv()`	`df.to_csv()`
json	`pd.read_json()`	`df.to_json()`
excel	`pd.read_excel()`	`df.to_excel()`
hdf	`pd.read_hdf()`	`df.to_hdf()`
sql	`pd.read_sql()`	`df.to_sql()`

Exercise